PECL 1 – Estructura de Datos

Rodrigo López Soriano – 09086605H
Rodrigo Martos Lagunda - 05958649Q

b. Detalles y justificación de la implementación

b.1. Especificación concreta de la interfaz de los TAD's implementados:

b.1.1. TAD's Creados

Para esta primera parte de la simulación hemos diseñado e implementado un conjunto de Tipos Abstractos de Datos (TADs) basados íntegramente en memoria dinámica (mediante el uso de nodos y punteros). Los TADs creados son:

- **TAD Paciente:** Es la estructura base de información. Almacena los datos personales y médicos generados (DNI, tipo de enfermedad, ID de prioridad y número de habitación asignado en el postoperatorio).
- **TAD Pila (y NodoPila):** Estructura LIFO (Last In, First Out) utilizada para almacenar a los pacientes recién llegados al hospital antes de ser clasificados en el triaje.
- **TAD Cola (y NodoCola):** Estructura FIFO (First In, First Out) que modela las salas de espera del preoperatorio (Salas A y B para apendicitis; Salas C y D para hernias).
- **TAD Lista (y NodoLista):** Lista enlazada simple diseñada específicamente para mantener a los pacientes ordenados por su ID (prioridad) mientras esperan entrar a los quirófanos.
- **TAD Controlador:** Actúa como el cerebro del programa. No es una estructura de datos convencional, pero es el gestor que contiene las instancias de las Pilas, Colas y Listas, y administra el flujo lógico de los pacientes (asignación de IDs únicos sin repetición, balanceo de colas y control del menú).

b.1.2. Definición de las operaciones del TAD (Nombre, argumentos y retorno).

A continuación se detallan los métodos públicos que definen la interfaz de cada estructura, indicando qué reciben (argumentos) y qué devuelven (retorno).

TAD Paciente

- Paciente(): Constructor por defecto. Argumentos: Ninguno. Retorno: Ninguno. Inicializa aleatoriamente el DNI y el tipo de enfermedad.
- tieneApendicitis(): Argumentos: Ninguno. Retorno: bool. Devuelve *true* si tiene apendicitis, *false* si tiene hernias.
- getId() / getHabitacion(): Argumentos: Ninguno. Retorno: int. Devuelven el ID o la habitación asignada.
- setId(int id) / setHabitacion(int habitacion): Argumentos: int. Retorno: void. Asignan los valores únicos durante el preoperatorio.
- mostrar(): Argumentos: Ninguno. Retorno: void. Imprime por consola los datos formateados del paciente.

TAD Pila

- insertar(Paciente v): Argumentos: Objeto Paciente. Retorno: void. Apila un nuevo nodo en la cima.
- extraer(): Argumentos: Ninguno. Retorno: Objeto Paciente. Saca y devuelve el paciente situado en la cima, liberando su memoria.
- cima(): Argumentos: Ninguno. Retorno: Objeto Paciente. Devuelve una copia del paciente en la cima sin sacarlo de la pila.
- getLongitud(): Argumentos: Ninguno. Retorno: int. Devuelve el número de elementos actuales.
- vaciar() / mostrar(): Argumentos: Ninguno. Retorno: void. Vacía la pila liberando memoria o la imprime por pantalla.

TAD Cola

- insertar(Paciente v): Argumentos: Objeto Paciente. Retorno: void. Encola un paciente al final de la estructura (puntero ultimo).
- eliminar(): Argumentos: Ninguno. Retorno: Objeto Paciente. Extrae y devuelve el paciente situado al frente de la cola (puntero primero).

- verPrimero(): Argumentos: Ninguno. Retorno: Objeto Paciente. Devuelve los datos del primer paciente sin extraerlo.
- getLongitud(): Argumentos: Ninguno. Retorno: int. Devuelve el tamaño de la cola para permitir el balanceo entre salas.
- vaciar() / mostrar(): Argumentos: Ninguno. Retorno: void. Limpia o imprime la cola.

TAD Lista (Ordenada)

- insertar(Paciente v): Argumentos: Objeto Paciente. Retorno: void. Inserta el paciente directamente en su posición correspondiente, comparando su ID con el resto de nodos para mantener la lista ordenada de mayor a menor prioridad.
- mostrarPrimero(): Argumentos: Ninguno. Retorno: void. Imprime el primer nodo de la lista (paciente con la prioridad más alta de esa lista).
- mostrarUltimo(): Argumentos: Ninguno. Retorno: void. Recorre la lista hasta el final e imprime el último nodo (paciente con la prioridad más baja).
- getLongitud() / vaciar() / mostrar(): Funcionalidad idéntica a las estructuras anteriores.

b.2 Solucion adoptada: descripción de las dificultades encontradas

Solución adoptada: Para afrontar la resolución de la práctica, hemos optado por un diseño modular basado plenamente en la Programación Orientada a Objetos. Hemos separado cada estructura de datos (Pila, Cola y Lista) en sus propios ficheros .hpp y .cpp, emparejados con sus respectivas clases de Nodos. Esto nos ha permitido aislar los errores y probar que cada estructura funcionaba bien de forma individual antes de unirlas.

En lugar de poner toda la lógica en el main.cpp, centralizamos todo el "cerebro" de la simulación en la clase Controlador. Esta clase actúa como un puente: contiene las instancias de las estructuras y es la encargada de extraer a los pacientes de una estructura y meterlos en otra según las opciones que el usuario elige en el menú interactivo.

Dificultades encontradas: A lo largo del desarrollo, nos enfrentamos a varios retos importantes, la mayoría derivados del uso de punteros y memoria dinámica en C++:

- **Fugas de memoria y pérdida de referencias:** Al principio, tuvimos problemas al hacer el método extraer() o eliminar(). Nos costó entender que antes de hacer un delete para borrar el nodo y liberar la memoria, teníamos que guardar obligatoriamente la dirección del siguiente nodo en un puntero auxiliar. Si borrábamos directamente, rompíamos la cadena y perdíamos toda la estructura. También tuvimos que ser muy meticulosos al programar los destructores para vaciar por completo las estructuras al cerrar el programa y no dejar "fugas de memoria".
- **Asignación de IDs y Habitaciones aleatorias pero únicas:** El enunciado exigía que las habitaciones y los identificadores fueran únicos dentro de unos rangos numéricos concretos. Si usábamos simplemente la función rand(), nos arriesgábamos a que salieran números repetidos (dos pacientes con la misma habitación). Para solucionar esta dificultad, decidimos crear unos vectores (arrays dinámicos) y llenarlos ordenadamente con todos los números posibles. Luego, usamos la librería de C++ para "barajar" esos vectores aleatoriamente, de forma que al asignar los datos simplemente extraíamos el último número del vector, garantizando total aleatoriedad sin ninguna repetición.
- **Inserción ordenada en las Listas:** Implementar las Pilas y las Colas fue relativamente sencillo porque siempre insertábamos en los extremos (cima o último). Sin embargo, insertar a los pacientes en los quirófanos ordenados por su ID nos resultó bastante más difícil. Tuvimos que plantear una lógica con dos punteros (aux y ant) para recorrer la lista buscando el hueco exacto donde encajaba el nuevo paciente, con mucho cuidado de reenganchar bien los punteros anterior y siguiente para no partir la lista por la mitad.

b.3 Explicación de los métodos más destacados

1. Inserción ordenada en la Lista (Lista::insertar) Este método es el algoritmo más complejo de nuestras estructuras de datos. A diferencia de las pilas o colas, aquí no insertamos simplemente al principio o al final. Usamos dos punteros auxiliares (aux y ant) para recorrer la lista con un bucle while. En cada iteración, comparamos el ID de prioridad del nuevo paciente con el ID del nodo actual. En el momento en que encontramos un paciente con un ID mayor, el bucle se detiene y reenganchamos los punteros para colocar al nuevo paciente exactamente en esa posición intermedia. También hemos incluido lógica condicional (if/else) para manejar casos especiales, como que la lista esté vacía o que el nuevo paciente tenga la máxima prioridad y deba ser la nueva cabeza de la lista.

2. Triaje y balanceo de salas (Controlador::opcionD_pasarAColas) Este método simula el núcleo lógico del preoperatorio. Su funcionamiento consiste en un bucle que va extrayendo pacientes de la pila principal hasta vaciarla. Por cada paciente, evalúa su tipo de enfermedad (apendicitis o hernias). En ese preciso instante, le asigna su ID y número de habitación extrayéndolos de nuestro sistema de números barajados. La parte más técnica es el balanceo dinámico: mediante la condición if (salaA.getLongitud() <= salaB.getLongitud()), el programa consulta en tiempo real qué cola tiene menos pacientes y lo inserta en ella. El uso del menor o igual (<=) nos garantiza además que, en caso de empate de aforo, el paciente entre por defecto a la sala A (o a la C en el caso de las hernias), tal y como exige el enunciado.

3. Generación de IDs únicos (Controlador::reiniciarNumerosAleatorios) Para cumplir con el requisito de que los identificadores y las habitaciones se asignen de forma aleatoria pero asegurando que jamás se repitan, diseñamos este método. Su funcionamiento consiste en rellenar cuatro std::vector con los rangos numéricos exactos permitidos (por ejemplo, del 1 al 49 para los IDs de apendicitis). Una vez los vectores están llenos y ordenados, utilizamos las herramientas de la librería <random> y la función std::shuffle para desordenarlos por completo en memoria. Gracias a esto, a la hora de asignar datos a un paciente, el sistema simplemente lee el último número del vector y lo destruye usando pop_back(). Es un método eficiente que nos da aleatoriedad sin necesidad de hacer bucles de comprobación de duplicados.

b.4 Explicación de los métodos más destacados

Al ejecutar el programa, se muestra por consola un menú interactivo que funciona mediante un bucle do-while. Este menú no se cierra hasta que el usuario introduce la letra 'S' (la entrada está protegida con la función toupper para que funcione aunque se escriba en minúsculas). En la parte superior de la pantalla, hemos implementado un "marcador" que muestra en tiempo real cuántos pacientes hay almacenados en cada estructura, lo que hace que la simulación sea mucho más visual.

El flujo lógico de la simulación sigue estos pasos:

1. **Llegada al hospital (Opción A):** El usuario genera pacientes de 12 en 12. El programa los guarda en la Pila siempre y cuando no se supere el límite de 48. En este punto, los pacientes "esperan" únicamente con su DNI y tipo de enfermedad asignados.
2. **Triaje y Preoperatorio (Opción D):** El programa simula el paso de los pacientes a las salas. Se vacía la pila por completo y los pacientes entran a las colas correspondientes (A y B para apendicitis, C y D para hernias). El comportamiento interno del programa revisa qué cola está más vacía para enviarlo allí. Justo en este paso, a cada paciente se le asigna de forma aleatoria su ID de prioridad y su número de habitación sin que se repita ninguno.
3. **Paso a Quirófano (Opción H):** Las salas de espera se vacían y los pacientes entran a las listas de los quirófanos. Como hemos implementado una inserción ordenada, al entrar en las listas quedan automáticamente ordenados por su ID de prioridad.
4. **Consultas e interacciones:** El resto de opciones (B, E, F, I, J) permiten al usuario consultar en cualquier momento el estado de las estructuras. Destaca la opción 'K', cuyo comportamiento es ir a las listas y extraer rápidamente a los pacientes ubicados en los extremos (el primero en una y el último en otra) para conocer las prioridades máxima y mínima. Por último, las opciones de borrado y reinicio se encargan de vaciar las estructuras llamando a los destructores para liberar la memoria dinámica.

b.5 Bibliografía

SIMULACIÓN DE TRIAJE Y ATENCIÓN A PACIENTES EN LOS QUIRÓFANOS DE UN HOSPITAL, **documento PDF de la PECL**

Diapositivas y material de apoyo de las sesiones prácticas impartidas en la asignatura de Estructuras de Datos.

https://cplusplus.com/

https://cppreference.com/